



Faculty of Computing

IT2021 - Artificial Intelligence and Machine Learning Project

Year 2 Semester 2 (2026)

Warehouse Personal Protective Equipment (PPE) Compliance Monitoring System

Group ID: WD-DS-01-G10

Group members

01	Kommilage S.G	IT24100641
02	Disanayaka S.K	IT24102217
03	Jayathunga N. P	IT24102981
04	Gamlath G.M.W.S	IT24102959
05	Athukorala M. B	IT24101937
06	Jayarathna W.A.S.H.S	IT24102968

26.04.2026
Date of submission

Table of contents

Confidentiality Notice.....	4
Introduction.....	5
1.1 Problem and Motivation	5
1.2 Literature Review.....	6
1.2.1 PPE Detection Using Deep Learning.....	6
1.2.2 YOLO Algorithms for Real-Time PPE Detection	6
1.2.3 Research Gaps Addressed by This Project	6
1.3 Aim and Objectives.....	7
1.3.1 Aim	7
1.3.2 Objectives	7
1.4 Solution Overview	7
Requirement Analysis.....	8
2.1 Stakeholder Analysis	8
2.2 Feasibility and SWOT Analysis.....	9
2.2.1 Feasibility Analysis.....	9
2.2.2 SWOT Analysis	10
2.3 Requirements Modelling.....	10
2.3.1 Functional Requirements	10
2.3.2 Non-Functional Requirements	11
2.3.3 Use Case Summary	12
Design and Development.....	18
3.1 System Architecture.....	18
3.1.1 User Flow diagram.....	20
3.2 AI/ML Component Design and Workflow	21
3.2.1 AI/ML Service Architecture	21
3.2.1.1 Data Pre-processing	21
3.2.1.2 Pre-trained Model Selection and Fine-Tuning.....	21
3.2.1.3 Client-Specific Dataset Annotation and Environmental Adaptation	23
3.2.1.4 Model Evaluation & Validation.....	24
3.2.1.5 Backend AI Service Layer	27
3.2.1.6 Web Integration and Unit Testing.....	27
3.3 Database Design and Data Management Strategy	27
3.3.1 Conceptual Data Model and Database Selection	27
3.3.2 Entity Relationships and Structural Design	28

Results and Evaluation.....	30
4.1 Overview of System Outcomes.....	30
4.2 Functional Testing and System Validation	30
4.3 AI/ML Feature Evaluation	36
4.3.2 Model Performance Metrics.....	36
4.3.3 Real-Time Inference Speed and Latency	36
4.3.4 False Positive Mitigation and Threshold Calibration.....	37
4.4 Non-Functional Evaluation	37
Deployment and Hosting Environment.....	37
5.1 Frontend Deployment	37
5.2 Backend Deployment.....	37
5.3 AI Component Deployment.....	38
Conclusion	38
6.1 Project Summary.....	38
6.2 Achievement of Objectives.....	38
6.3 Lessons Learned.....	38
6.4 Limitations and Future Work.....	39
APPENDIX A — TEAM CONTRIBUTION TABLE.....	40

Confidentiality Notice

This report contains confidential information relating to the client organisation. The contents of this report, including system designs, operational data, and implementation details, are intended solely for academic assessment purposes. All client-identifying information has been anonymised. This document must not be distributed, reproduced, or disclosed to any third party without prior written consent.

Introduction

1.1 Problem and Motivation

Warehouses are naturally dangerous places due to their use of large equipment, moving vehicles, and constant manual movement of products. Workers in a warehouse environment are exposed to risks that may lead to serious injury or death without the proper use of Personal Protective Equipment (PPE), including hard hats, high-visibility vests, and face masks. They are at the highest risk of being injured when they do not consistently wear PPE. PPE is the main form of protection against many hazards found in the work environment.

However, ensuring compliance with PPE on an active warehouse floor remains challenging due to traditional methods of monitoring employees. These methods depend on having a safety officer or floor manager present to conduct on-site inspections of employees at set times throughout the day. Manual monitoring is necessary to keep up with PPE requirements, but it does not provide an efficient or cost-effective way to monitor operations. The time needed for manual inspection makes it impossible for supervisors to monitor every employee in real time. Conducting inspections at fixed intervals instead of continuously leading to gaps in real-time data. This creates situations where an employee might violate PPE regulations without being documented or addressed.

In addition to the limitations of monitoring coverage, manual monitoring creates further problems. It often leads to inconsistent enforcement due to fatigue or lack of focus. There is also no centralized data for trend analysis, which makes safety management more reactive rather than proactive. Many reports on incidents and compliance are typically prepared manually, creating additional administrative workload and making it difficult to collect and analyze data for decision-making. As a result, there is a higher risk of workplace accidents, reduced operational efficiency, and greater difficulty in demonstrating compliance with safety regulations to regulators or clients.

The objective of this project is to create an automated and data-driven system for continuous monitoring of PPE compliance, logging violations as they occur, and providing management with actionable safety intelligence. This system aims to replace the current manual and reactive approach to workplace safety management. The implementation of modern deep learning-based object detection technologies, such as YOLO, within warehouse safety processes has strong potential to transform safety management from a human-dependent and periodic model into one that is real-time, intelligent, and scalable.

1.2 Literature Review

The application of computer vision to workplace safety monitoring has gained significant attention over the past decade. This section reviews key developments in Personal Protective Equipment (PPE) detection using deep learning, with a particular focus on YOLO-based architectures, and highlights the research gaps addressed in this project.

1.2.1 PPE Detection Using Deep Learning

Early approaches to PPE detection leveraged Convolutional Neural Networks (CNNs) to identify basic safety equipment such as helmets and high-visibility vests from construction site images. These methods demonstrate that automated PPE compliance monitoring is feasible using computer vision techniques. However, such systems were often limited in scope, focusing on only a small number of PPE categories and lacking the capability to process live video streams, which are essential for continuous, real-time monitoring in industrial environments.

Subsequent advancements expanded detection capabilities to include multiple PPE categories simultaneously, such as helmets of different types, safety vests, and protective footwear. While these systems improved multi-class detection performance, they were typically designed as standalone detection tools. As a result, they did not address broader operational requirements such as integration with safety management systems, violation tracking, inventory monitoring, or automated compliance reporting.

1.2.2 YOLO Algorithms for Real-Time PPE Detection

The YOLO (You Only Look Once) family of object detection models has become a leading approach for real-time PPE compliance monitoring due to its strong balance between speed and accuracy. Unlike traditional two-stage detectors that separate region proposal and classification, YOLO performs both tasks in a single forward pass, enabling high-speed inference suitable for real-time applications.

Recent developments in YOLO architectures have further improved detection performance and efficiency. Various versions have been evaluated for PPE detection tasks, demonstrating differences in accuracy and inference speed across model variants. While some models achieve higher detection accuracy, others provide faster inference times, making them more suitable for real-time deployment scenarios.

Overall, these advancements highlight the importance of selecting a model that balances accuracy with computational efficiency. In practical industrial applications, real-time performance and deployment feasibility are often as critical as detection accuracy, particularly in environments requiring continuous monitoring and immediate safety interventions.

1.2.3 Research Gaps Addressed by This Project

While the above studies prove that YOLO-based models can detect PPE accurately, most prior work focuses solely on the detection model itself. In contrast, real-world deployment requires a complete operational ecosystem. Literature lacks examples of integrating PPE detectors with a centralized safety platform or connecting detections to inventory management and compliance dashboards. Likewise, few systems support two-way safety feedback (e.g. allowing workers to query or receive alerts). Existing studies often assume ideal imaging conditions (well-lit environments, fixed camera angles) that may not hold in a busy warehouse or logistics setting. Our project explicitly addresses these gaps by building a full PPE monitoring system: we will embed a fast YOLOv8-based detector into a management platform that handles inventory tracking, violation logging, and automated reporting, all tailored to the challenging conditions of real

industrial sites

1.3 Aim and Objectives

1.3.1 Aim

This project will create a system for tracking personal protective equipment (PPE) compliance in warehouses using artificial intelligence and will introduce an automated way of detecting violations of PPE rules in real time, will create a single point of management for safety data, will establish an integrated solution for managing inventory, reporting on compliance and engaging employees in safety, and all of these functions will replace the way safety was previously monitored manually with a more proactive and data driven safety system.

1.3.2 Objectives

- Objective 1 – Automate Compliance Monitoring: Replace manual inspections with continuous AI-driven monitoring using a YOLOv8-based detection model to ensure consistent PPE regulation enforcement across all warehouse zones.
- Objective 2 – Enhance Adherence to Workplace Safety Standards: Enable real-time detection of safety violations, specifically the absence of required PPE items including safety helmets and high-visibility vests.
- Objective3 – Provide Safety Insights for Management Decisions: Develop centralized dashboards and automated safety compliance reports to support management decision-making with accurate, up-to-date safety data.
- Objective 4 – Improve Operational Efficiency: Deploy a lightweight, cost-effective YOLOv8-based detection service that performs inference without requiring significant hardware investment, maintaining real-time processing capability.
- Objective 5 – Track and Manage Safety Equipment Inventory: Implement an inventory management module to track PPE stock levels, assign equipment to employees, and support procurement decisions.
- Objective 6 – Establish an Integrated Safety Ecosystem: Unify AI monitoring, compliance reporting, inventory management, and employee interaction into a single scalable platform, enabling employees to submit inquiries and view their assigned equipment.

1.4 Solution Overview

This document outlines a Warehouse Personal Protective Equipment (PPE) Compliance Monitoring System that has been developed for Hayleys Advantis 3PL Plus (Logiwiz Limited). The system consists of six major functional components that will come together as part of one system: Warehouse Configuration, Safety Equipment Inventory Management, AI Model Evaluation (and) Inference, Employee Management and Feedback, Safety Operations and Compliance Reporting, and Notification and Reporting Management. At its core, the system employs a YOLOv8-based computer vision model to perform real-time PPE detection from CCTV video streams captured via RTSP protocol. Detected violations are automatically logged to a MongoDB database, where they inform compliance dashboards, trigger automated notifications, and contribute to daily and monthly safety reports accessible to warehouse management. The frontend is developed using React with Vite and Material UI (MUI), providing role-specific interfaces for warehouse configuration managers, inventory managers, AI/ML engineers, safety reporting officers, and individual employees. The backend is built on Node.js with Express, while the AI inference service is implemented

as a standalone FastAPI microservice using Python, OpenCV, and the Ultralytics YOLO library. Media files including violation snapshot images are stored in cloud object storage (Clouinary).

The Git repository for this project is available at the following link:

Project Repository: <https://github.com/IT2021-AIML-Project/Warehouse-Safety-Monitoring-Full-System.git>

Note: Please replace the above URL with the actual Git repository link before submission.

Requirement Analysis

This chapter presents the requirement analysis conducted for the Warehouse PPE Compliance Monitoring System. It covers the identification of key stakeholders and their needs, a feasibility study and SWOT analysis, and the structured modelling of system requirements.

2.1 Stakeholder Analysis

Stakeholder analysis was conducted to identify all individuals and groups with a direct interest in the system, understand their roles, needs, and concerns, and ensure that the system design aligns with real operational requirements. The following table summarises the key stakeholders identified for this project.

Stakeholder	Role	Needs / Interests	Concerns
Warehouse Configuration Manager	Manages zones, cameras, and employee records	Accurate warehouse configuration; ability to add/modify zones and cameras	System usability; data accuracy
Inventory Manager	Oversees PPE stock, assignment to employees	Real-time stock visibility; accurate assignment records	Stock shortages; mis-assignment
Safety / Reporting Officer	Reviews violation reports and compliance scores	Automated reports; zone-level compliance data; trend analysis	Report accuracy; timely alerts
AI / ML Engineer	Manages model inference sessions; reviews detection logs	Model performance metrics; violation detection accuracy	False positive rates; inference latency
Warehouse Employee	Subject of PPE compliance monitoring	View assigned PPE; submit inquiries; receive safety notifications	Privacy; fairness of monitoring
Senior Management (Logiwiz Limited)	Decision-makers responsible for safety outcomes	High-level compliance dashboards; monthly safety summaries	Cost of incidents; regulatory compliance
System	Maintains system	System stability; role-based	Security; data

Administrator	infrastructure and user accounts	access control	integrity
---------------	----------------------------------	----------------	-----------

Table 2.1: Stakeholder Analysis

2.2 Feasibility and SWOT Analysis

2.2.1 Feasibility Analysis

A feasibility study was conducted across four dimensions technical, operational, economic, and legal to confirm the viability of the proposed system prior to development.

Technical Feasibility: The system relies entirely on proven, open-source technologies. YOLOv8 by Ultralytics is a well-established object detection framework with documented performance in PPE detection contexts. The MERN stack (MongoDB, Express.js, React, Node.js) is a widely adopted web technology combination capable of supporting the full-stack requirements of the platform. Python with OpenCV and FastAPI provides a robust environment for the AI inference microservice. CCTV integration via RTSP protocol is a standard industrial approach, and cloud storage solutions such as AWS S3 are commercially available and well-documented. All component technologies have been validated in prior academic and industry projects, confirming technical feasibility.

Operational Feasibility: The system is designed so that it will reduce the operational impact to each of the user groups. Updates to the warehouse configuration may only occur during major infrastructure events such as installing new CCTV cameras or adding additional zones. The AI inference service operates independently and does not require technical knowledge of the end user. Employee side interfaces are browser-based and require minimal training to use. Management dashboards provide for automated aggregations, thus eliminating the need to manually prepare reports. Alerts are sent to communication systems (e.g., Email and Dashboards) as part of an overall notification system and are designed to work inherently with existing Management processes.

Economic Feasibility: The system is built entirely using open-source software, eliminating licensing costs. Datasets used for model training are publicly available through Roboflow and Kaggle. Pre-trained YOLO model weights are freely available via Ultralytics. Cloud storage and deployment costs are minimized through structured data management, MongoDB stores structured metadata while only violation snapshot images are stored in object storage, following a tiered cost model. No specialized hardware beyond existing CCTV infrastructure is required for deployment.

Legal Feasibility: The system is designed to store structured detection metadata, bounding box coordinates, class labels, confidence scores, and timestamps, rather than retaining raw CCTV footage. This approach aligns with data minimization principles applicable to surveillance systems. The use of publicly available datasets for model training ensures no client data is used during the development phase. The system is positioned as an enhancement to existing CCTV-based monitoring practices already in operation at the client facility, reducing the risk of regulatory friction.

2.2.2 SWOT Analysis

A SWOT analysis was conducted to evaluate the internal and external factors affecting the project. The analysis is presented in the table below.

Strengths	Weaknesses
1. Modular Microservice Architecture: Clear separation between frontend, backend, AI inference service, database, and storage enables scalability, maintainability, and future extensibility. 2. AI-Driven Automation: YOLO-based computer vision enables automated safety monitoring, reducing reliance on manual supervision. 3. Configurable Monitoring Strategy: The system supports scheduled or event-driven inference rather than continuous full-day surveillance, optimising computational resource usage. 4. Scalable Data Storage: Separation of structured data (MongoDB) and media files (object storage) ensures stable performance as violation data grows over time.	1. Dependence on Visual Data Quality: System performance is highly sensitive to lighting conditions, camera angles, and environmental factors such as occlusion by shelving or forklifts. 2. Hardware and Computational Dependency: Real-time AI inference requires adequate GPU or CPU resources, which may constrain deployment in resource-limited environments. 3. Limited Contextual Understanding: Computer vision can detect visible PPE but cannot fully interpret behavioural context — for example, a worker holding a helmet rather than wearing it may not be reliably flagged.
Opportunities	Threats
1. Expansion Beyond Warehouse Environments: The platform architecture can be adapted for construction sites, manufacturing plants, laboratories, and healthcare settings with minimal modification. 2. Multi-Site Deployment: The modular design enables scaling to multiple warehouse facilities with a centralized compliance monitoring hub. 3. Integration with Predictive Analytics: Historical violation data can be used to identify high-risk zones, predict compliance trends, and optimize workforce safety training programmes. 4. SaaS Commercialisation: Cloud-based deployment enables transformation into a subscription-based safety compliance platform for multiple clients.	1. Privacy and Ethical Regulations: Surveillance-based systems may face regulatory restrictions depending on jurisdiction and employee consent requirements. 2. Cloud Storage and Operational Cost Fluctuation: Long-term storage of violation snapshot data can increase operational costs if a structured log retention policy is not enforced. 3. Network Dependency for Real-Time Monitoring: Unstable network connectivity between CCTV cameras and the AI inference service can affect live stream processing and introduce detection latency.

Table 2.2: SWOT Analysis of the Warehouse PPE Compliance Monitoring System

2.3 Requirements Modelling

2.3.1 Functional Requirements

The system comprises six primary functional components, each addressing a distinct area of the warehouse

safety management workflow. The functional requirements for each component are summarized below.

Component	Description	Key Functions	Status
Warehouse Configuration System	Manages the structural foundation of safety monitoring including warehouse zones, employee records, and CCTV input configuration.	Create/Update/Delete zones, cameras, and employee records	Finished
Safety Equipment Inventory Management	Handles PPE type registration, stock tracking, assignment to employees by zone, and inventory lifecycle management.	Add/View/Update/Delete PPE items and zone assignments	Finished
AI Model Evaluation and Inference	Responsible for AI inference execution, violation detection logging, model confidence tracking, and inference session management.	Run inference; log detections and violations; manage sessions	Finished
Employee Management and Feedback System	Manages employee records, View employee assigned zone, and provides a feedback portal where employees can submit inquiries/feedbacks and view their assigned PPE.	Employee login; submit/view/edit/delete inquiries; submit/view feedbacks,view assigned PPE	Finished
Safety Operations and Compliance Reporting	Generates daily and monthly violation reports, section-wise compliance scores, and safety dashboards for management review.	Generate reports; view compliance dashboards; filter by zone/date	Finished
Notification System and Reporting	Handles real-time violation notifications sent to supervisors and scheduled monthly compliance reports delivered via email and dashboard alerts.	Generate, send, view, and manage notifications and alerts	Finished

Table 2.3: Functional Requirements Summary

2.3.2 Non-Functional Requirements

In addition to functional capabilities, the system must satisfy the following non-functional requirements to ensure quality, reliability, and operational suitability.

- ❑ **Security:** The system implements secure JWT-based authentication and role-based access control (RBAC) to ensure that each user type (Warehouse Configuration Manager, Inventory Manager, AI Engineer, Reporting Officer, Employee) can only access features relevant to their role. All API communications are secured, and sensitive user data is encrypted.
- ❑ **Performance:** The system is designed to provide responsive dashboards with minimal latency. The AI inference microservice processes CCTV frames efficiently using YOLOv8, targeting real-time throughput suitable for warehouse monitoring.
- ❑ **Scalability:** The microservice architecture enables the system to scale horizontally. Additional

CCTV cameras, warehouse zones, and users can be added without architectural changes. The separation of MongoDB for structured data and cloud object storage for media files ensures stable performance as data volumes grow.

- Reliability: This system has been designed to endure through stable and continuous operation with minimal amounts of downtime. Through asynchrony of API processing and the batching of inference logs, the system will reduce the likelihood of creating bottlenecks while processing.
- Scheduling and Automation: The system supports automated report scheduling, enabling daily safety summaries and monthly compliance reports to be generated and distributed without manual intervention.

2.3.3 Use Case Summary

The six component-level use cases that were created to illustrate the functional requirements of the various components of the system and represent their corresponding components in the system have been created to support the actor interactions, which include the Warehouse Configuration Manager, Inventory Manager, AI Engineer, Reporting Officer, Employee and System Administrator. The use case diagrams for these use cases can be found in Chapter 3 as part of the design documentation.

Each use case is structured with primary and extension points. For example, within the Employee Management and Feedback System, the Submit Inquiry use case includes extension points for editing and deleting inquiries where the inquiry status is open. Within the Safety Equipment Inventory Management component, the Manage Inventory use case extends to support adding, updating, viewing, and deleting PPE items. This structured approach to use case modelling ensures that all identified functional requirements are traced to specific system behaviors.

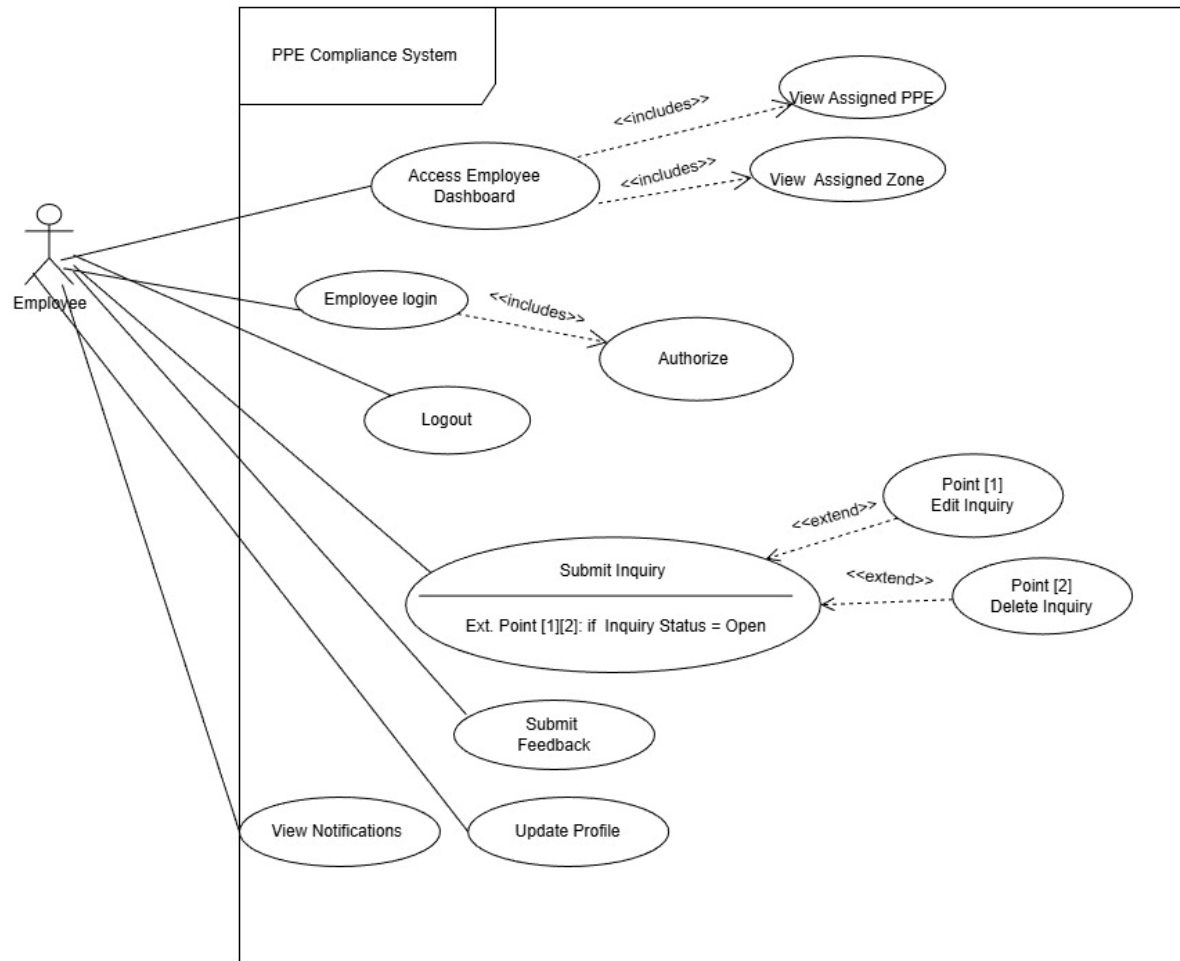


Figure 2.3.3.1: Use case diagram of Employee Management and Feedback System

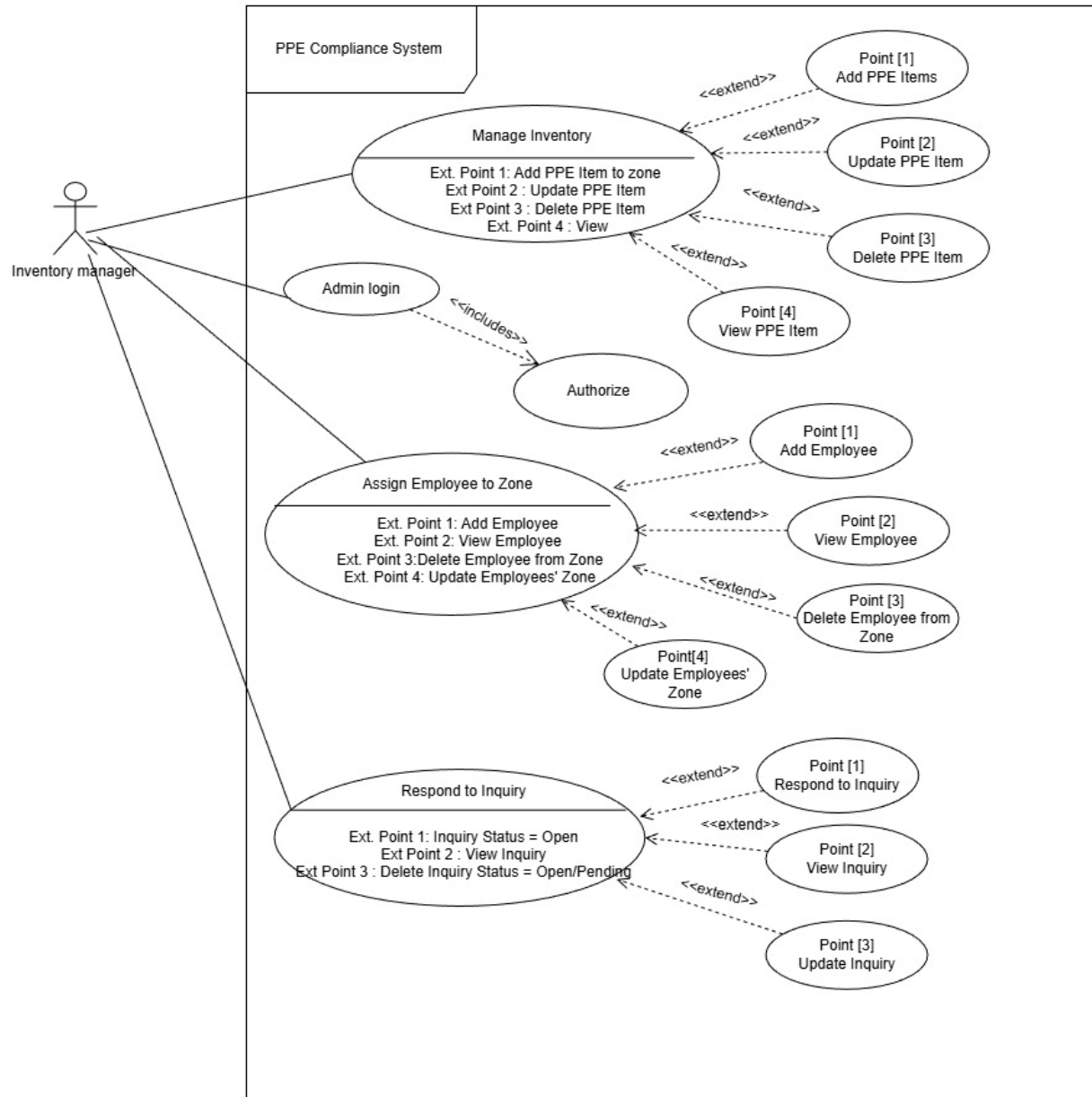


Figure 2.3.3.2: Use case diagram of Safety Equipment Inventory Management

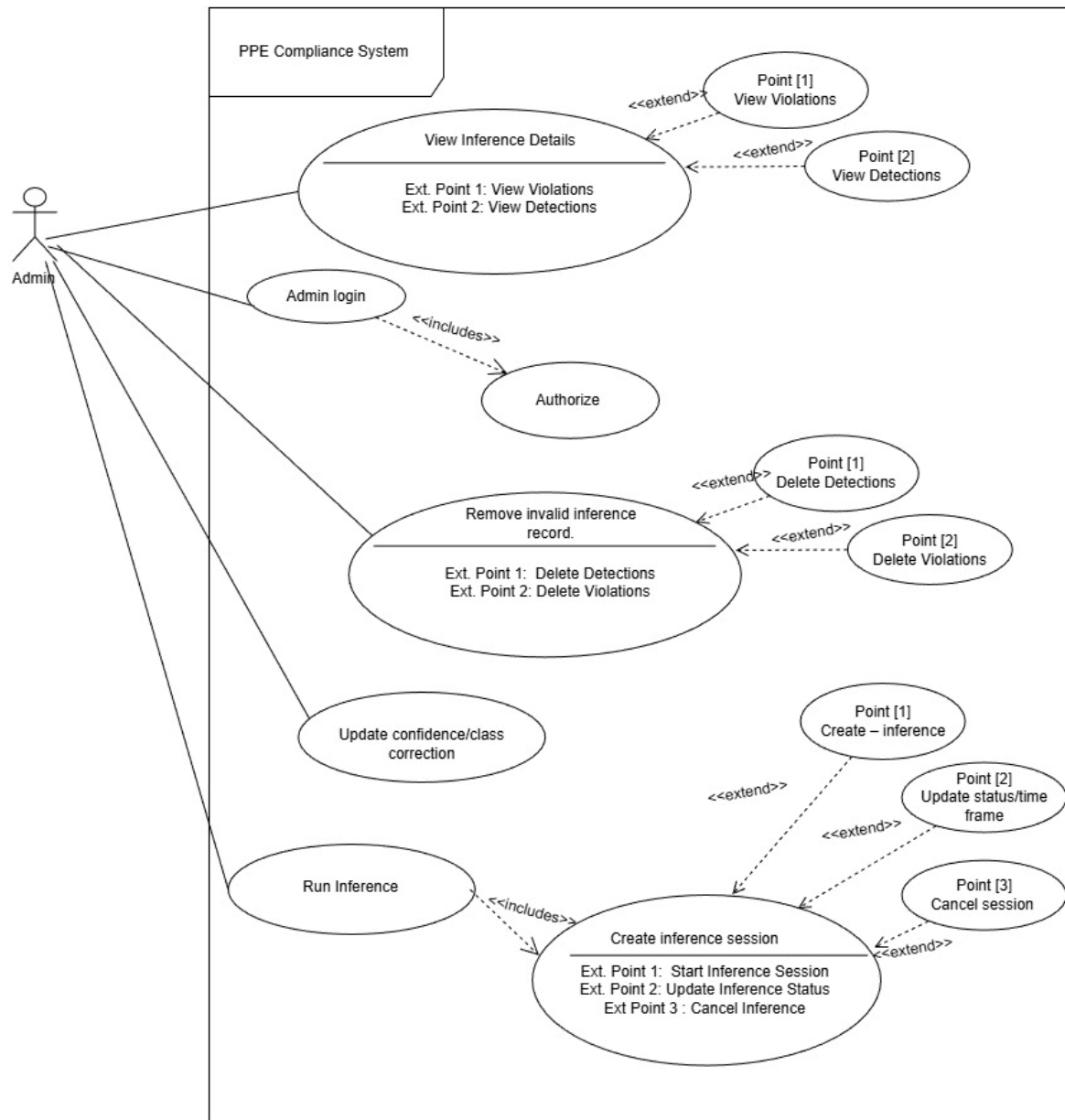


Figure 2.3.3.3: Use case diagram of Model Performance Monitoring and Governance

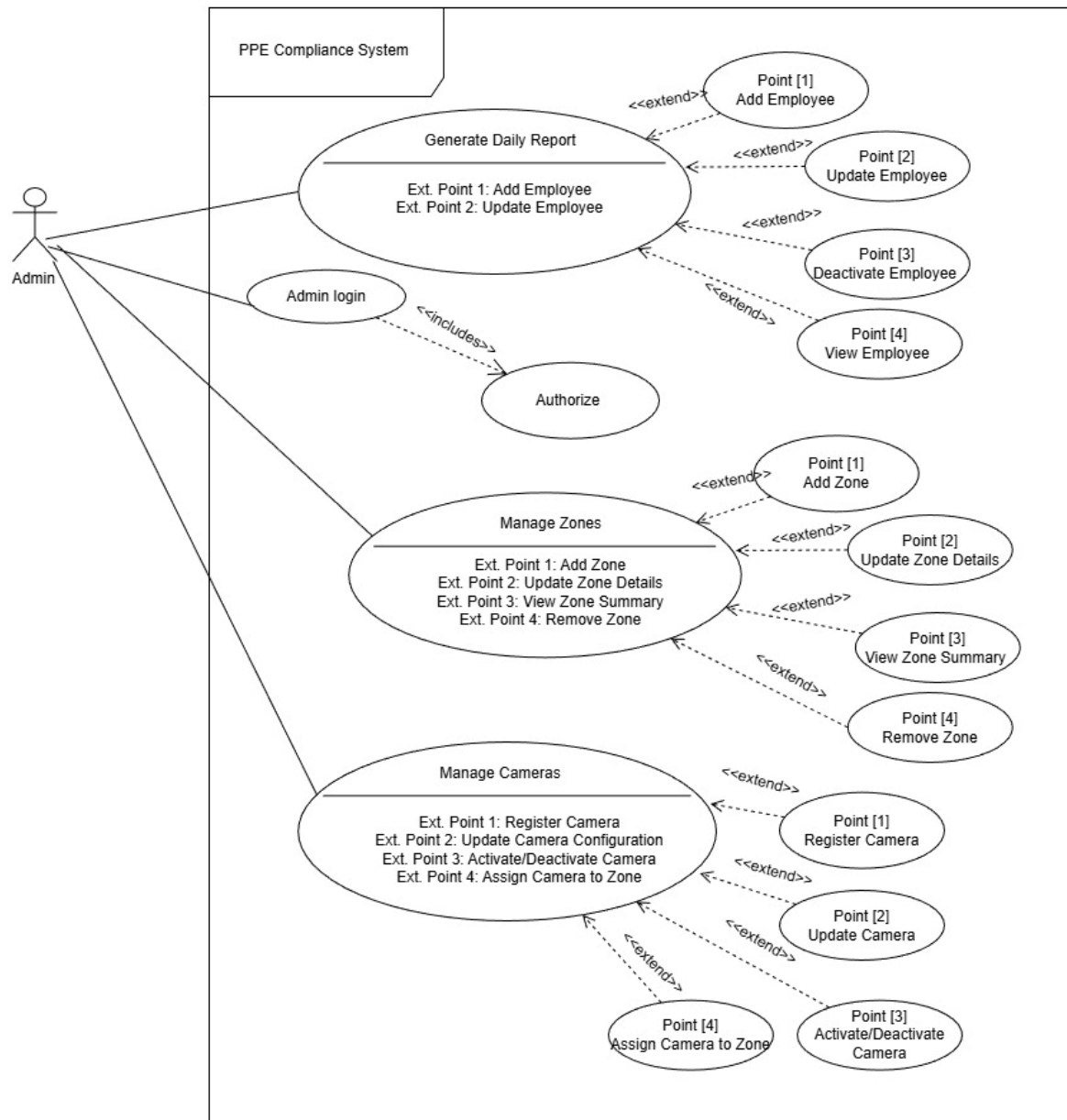


Figure 2.3.3.4: Use case diagram of Safety Operations and Compliance Reporting

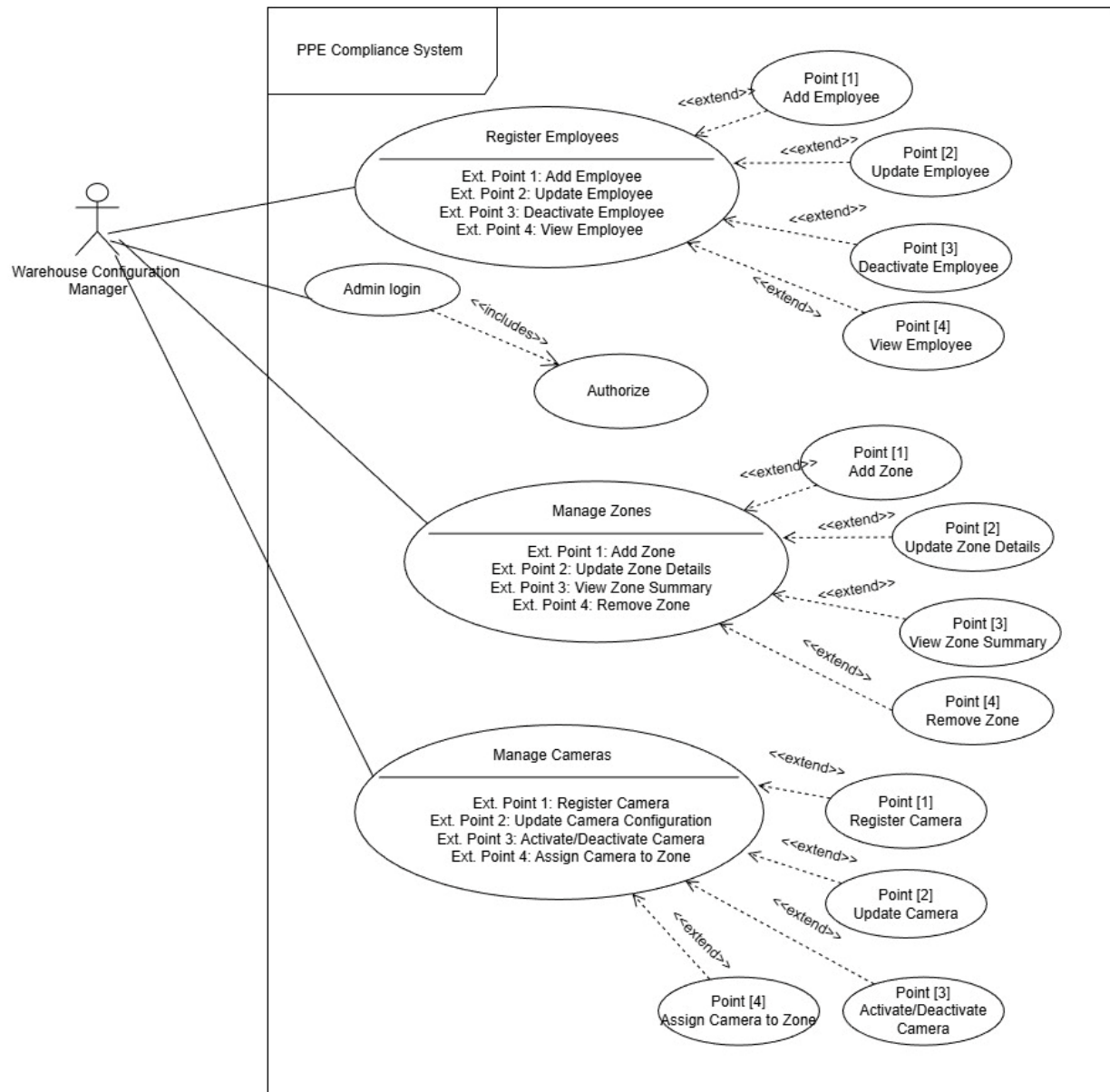


Figure 2.3.3.5: Use case diagram of Warehouse Configuration System

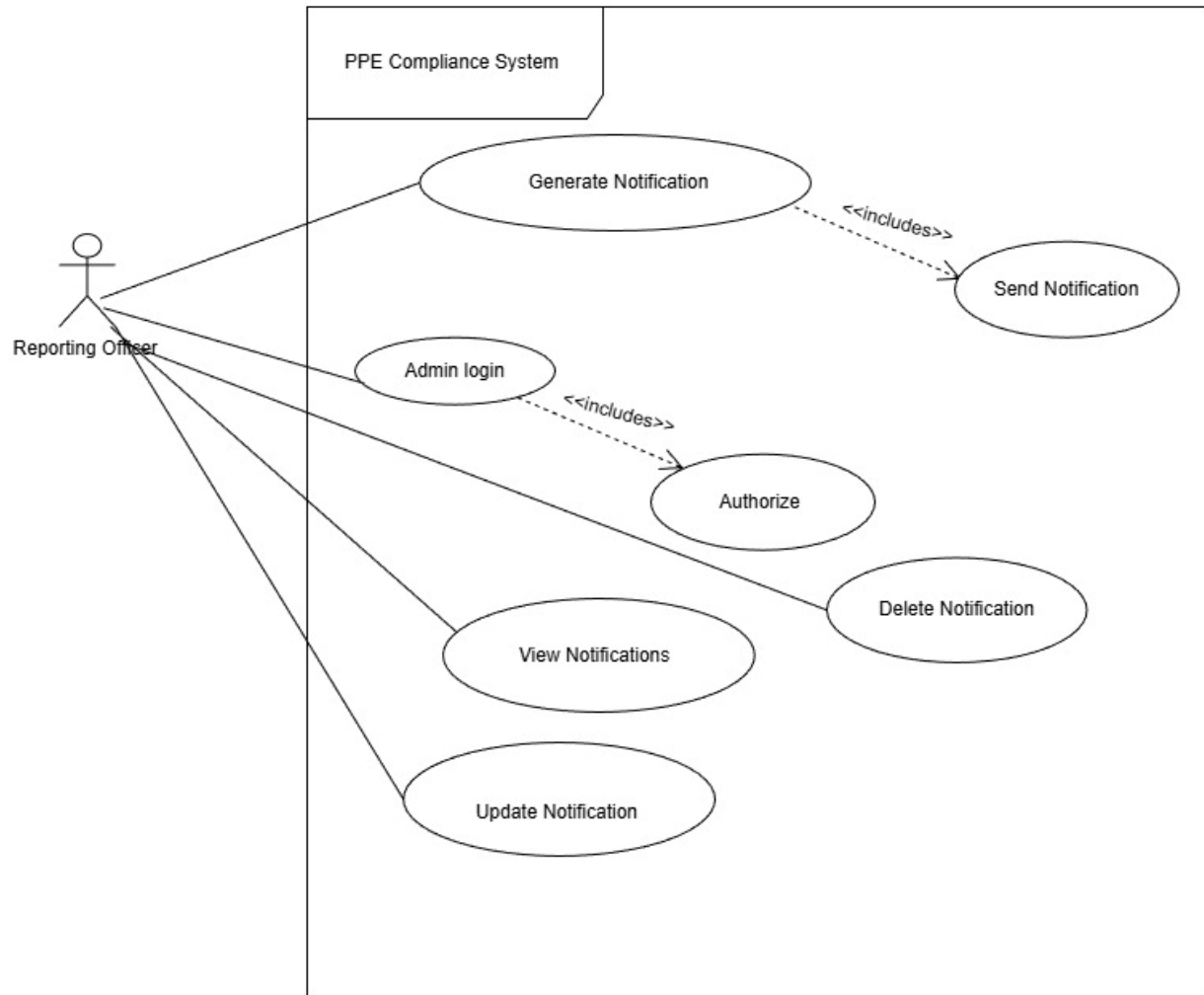


Figure 2.3.3.6: Use case diagram of Notification System and Reporting

Design and Development

3.1 System Architecture

The proposed system is built upon a modular, microservice-oriented architecture designed to ensure scalability, maintainability, and real-time performance. The architecture is logically divided into several interconnected layers:

- **Presentation Layer (Frontend):** The user interface is developed as a React Web Application utilizing Vite and Material UI (MUI). It provides distinct, role-based dashboards for model Performance monitoring, warehouse configuration, inventory management, employee feedback, and compliance reporting.
- **Application Layer (Backend):** Business logic is handled by a Node.js runtime environment as

well as an Express.js framework. This layer serves as a middle point for communication between the Frontend (User Interface), Database, and AI service through secure RESTful APIs.

- **AI/ML Inference Layer:** A dedicated, isolated microservice is implemented using Python and exposed via FastAPI. This service connects to warehouse CCTV cameras utilizing the Real-Time Streaming Protocol (RTSP) and captures video frames via OpenCV. The extracted frames are then processed by the YOLOv8 (Ultralytics) model to execute real-time object detection for PPE compliance.
- **Data and Storage Layer:** A hybrid approach to data management, using MongoDB as the primary NoSQL database for managing structured records such as user profiles, equipment inventory, and annotated detection logs. Heavy and unstructured media files such as violation snapshots and video clips are stored in scalable Cloud/AWS Object Storage solutions (S3 buckets) to allow for optimized performance.

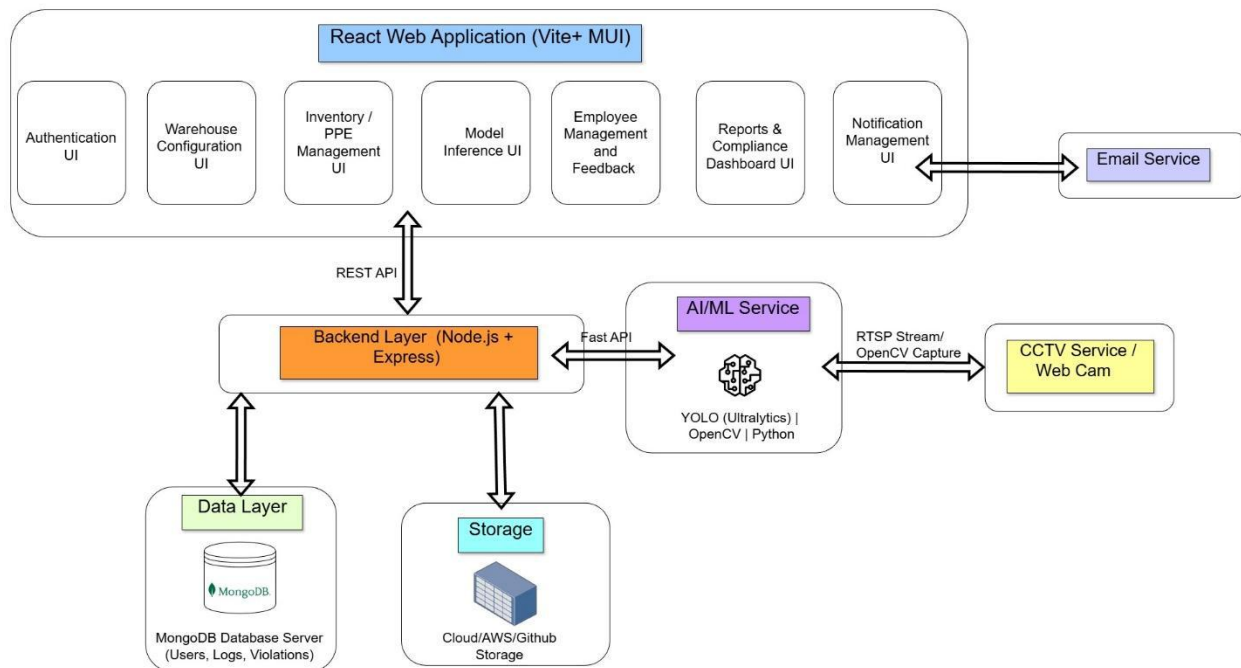
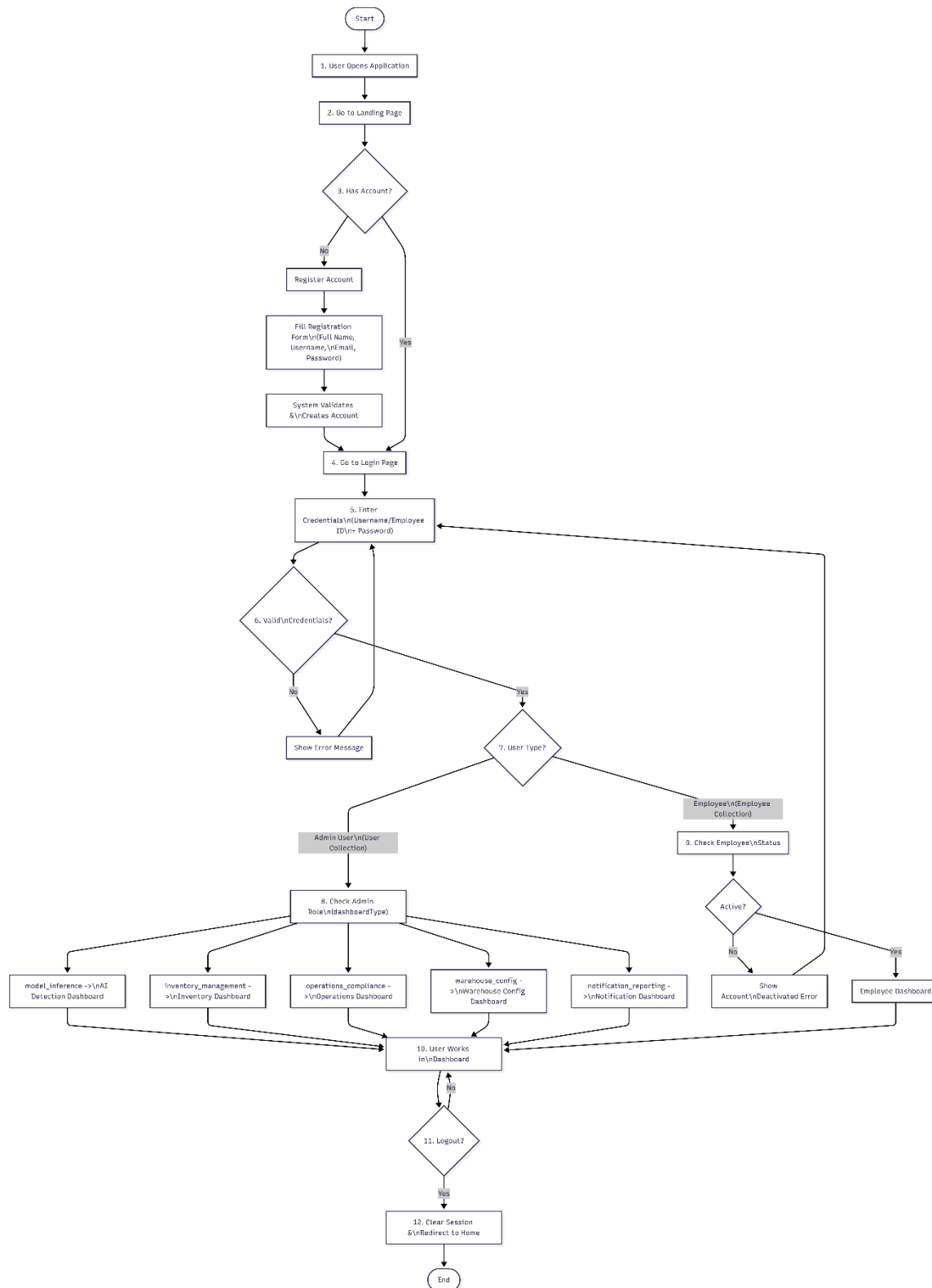


Figure 3.1: System Architecture Diagram

3.1.1 User Flow diagram



3.2 AI/ML Component Design and Workflow

3.2.1 AI/ML Service Architecture

The AI/ML component is engineered as an independent, decoupled microservice utilizing FastAPI. This modular approach ensures that the computationally intensive object detection tasks do not bottleneck the main application. The service captures real-time CCTV video streams via the Real-Time Streaming Protocol (RTSP) and extracts individual frames utilizing OpenCV. The core object detection is powered by the YOLOv8 (Ultralytics) model, selected for its optimal balance of high-speed inference and accuracy, which is crucial for real-time surveillance environments.

3.2.1.1 Data Pre-processing

The foundation of the AI model's reliability depended on high-quality data engineering. This stage involved aggregating datasets from platforms like Roboflow, cleaning incorrect annotations, and standardizing all bounding boxes into the strict YOLO format. Synthetic perturbations such as brightness adjustments, simulated motion blur, and geometric transformations (e.g., slight rotations) were applied to ensure the dataset accurately reflected the dynamic reality and low-light conditions of an industrial warehouse. All images were uniformly resized to 640 $\times$ 640 pixels, resulting in a versioned, production-ready dataset.

3.2.1.2 Pre-trained Model Selection and Fine-Tuning

Prior to training, a comprehensive evaluation of candidate model architectures was conducted to identify the most suitable baseline for PPE detection. Different variants of the Ultralytics YOLOv8 family, including lightweight and small-scale models, were compared based on key trade-offs such as model size, computational requirements, inference latency, and detection accuracy. This selection process ensured that the chosen model achieves an optimal balance between efficiency and performance, making it suitable for deployment on standard edge hardware while maintaining the ability to process real-time CCTV video streams without frame drops.

Following model selection, the chosen YOLOv8 architecture was fine-tuned using transfer learning techniques on a general PPE dataset to adapt it to the target detection task. Training involved configuring dataset parameters and performing systematic hyperparameter tuning, including adjustments to learning rate, batch size, and number of epochs.

To ensure training stability and strong generalization, techniques such as checkpointing and early stopping were applied. These mechanisms enabled continuous monitoring of model performance and helped prevent overfitting, resulting in a robust and efficient baseline PPE detection model suitable for real-time deployment.

Model	Params	Inference (ms)	mAP@0.5
YOLOv8n (Selected in NVIDIA GeForce GTX 1650)	3.2M	~22ms	0.837
YOLOv8s	11.2M	~128.4ms	0.943

Model	Params	Inference (ms)	mAP@0.5
YOLOv9s	7.2M	~116 ms	0.95
YOLOv11s	9.4M	~90.0 ms	0.950

The selected YOLOv8n architecture was adapted to the warehouse-specific PPE dataset through transfer learning, initialising from Ultralytics pre-trained weights trained on the COCO dataset. The training configuration used a batch size of 16, a learning rate schedule with linear warm-up over the first 3 epochs, and a total of 50 training epochs. Early stopping with a patience of 15 epochs was enabled to prevent overfitting, alongside model checkpoint versioning to retain the best-performing weights by validation mAP@0.5.

Figure 3.2 presents the training and validation loss curves alongside key performance metrics across all 50 epochs. All three training loss components — bounding box regression loss (box_loss), classification loss (cls_loss), and distribution focal loss (dfl_loss) — exhibit consistent monotonic descent, converging smoothly without instability or plateau, which indicates that the learning rate schedule and batch configuration were well-suited to the dataset size. Validation losses follow the same downward trend and remain closely parallel to the training losses throughout training, confirming that the model generalises well to unseen data without overfitting. Validation mAP@0.5 reaches approximately 0.84 by epoch 50, with precision stabilising above 0.90 and recall continuing to improve in the final epochs, suggesting the model had not yet fully converged and that additional training epochs could yield marginal further gains.

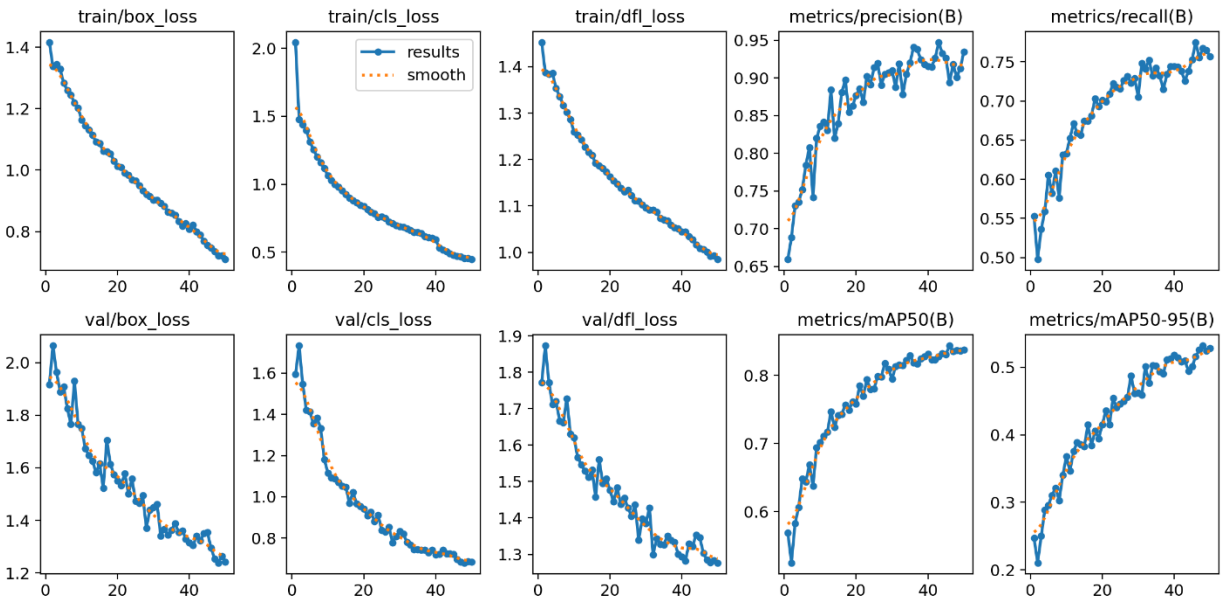


Figure 3.2: YOLOv8n training and validation loss curves with precision, recall, and mAP metrics across 50 epochs.

3.2.1.3 Client-Specific Dataset Annotation and Environmental Adaptation

To further improve model performance under real-world deployment conditions, a small client-specific dataset was collected from the target warehouse environment. This dataset captured unique environmental characteristics such as camera angles, lighting conditions, worker density, and background clutter that are not fully represented in generic PPE datasets.

Due to its limited size, the client dataset was not sufficient for full model training. Instead, it was manually annotated and used for targeted fine-tuning of the pre-trained model. Annotation was performed using bounding boxes for relevant classes, including persons, safety helmets, and safety vests, ensuring consistency with the primary training dataset.

This additional fine-tuning stage enabled the model to adapt to domain-specific variations, improving detection accuracy and robustness in the client's operational environment. By incorporating real-world visual conditions into the training pipeline, the system achieves better generalization during deployment, particularly in challenging scenarios such as partial occlusion, varying illumination, and dense warehouse layouts.



Figure : Client data

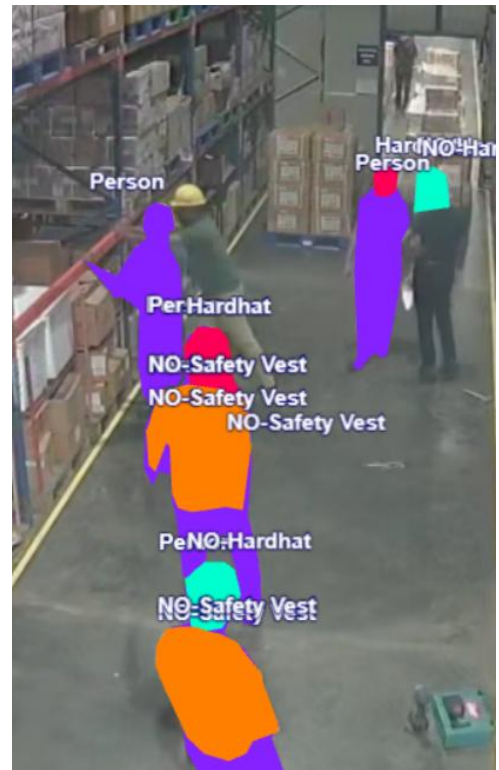


Figure : Client data annotated

3.2.1.4 Model Evaluation & Validation

To scientifically verify the model's reliability for production, comprehensive risk assessments and statistical evaluations were performed on unseen validation data. The model was benchmarked using key performance metrics, analyzing the Precision-Recall curve, F1-Confidence curves, and Mean Average Precision (mAP50-95). Extensive confusion matrix analysis and threshold calibration were conducted to minimize false positives, ensuring the false alarm rate was operationally acceptable for warehouse management, alongside FPS benchmarking to validate real-time capability.

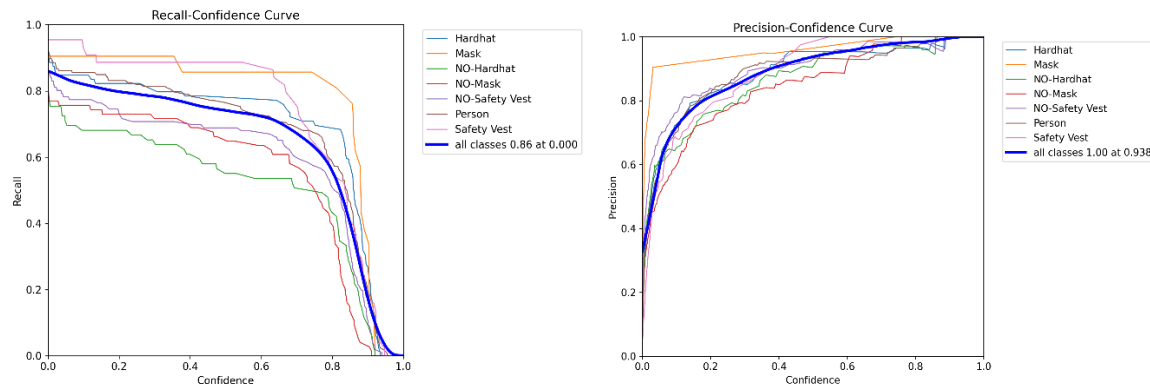


Figure : Precision-Confidence curve and Recall-Confidence curves

The figure above presents the Precision-Confidence and Recall-Confidence curves. Precision reaches 1.00 at high confidence levels (around 0.938), indicating that detections are highly accurate when strict thresholds are applied. At the same time, the model achieves an overall recall of 0.86 at zero confidence, showing strong detection capability before filtering.

As confidence increases, recall decreases, with the NO-Hardhat class showing the most significant drop, making it the most prone to missed detections. Based on these trends, a confidence threshold of 0.60 was selected as it provides a balanced trade-off, maintaining high precision while preserving sufficient recall for effective safety monitoring.

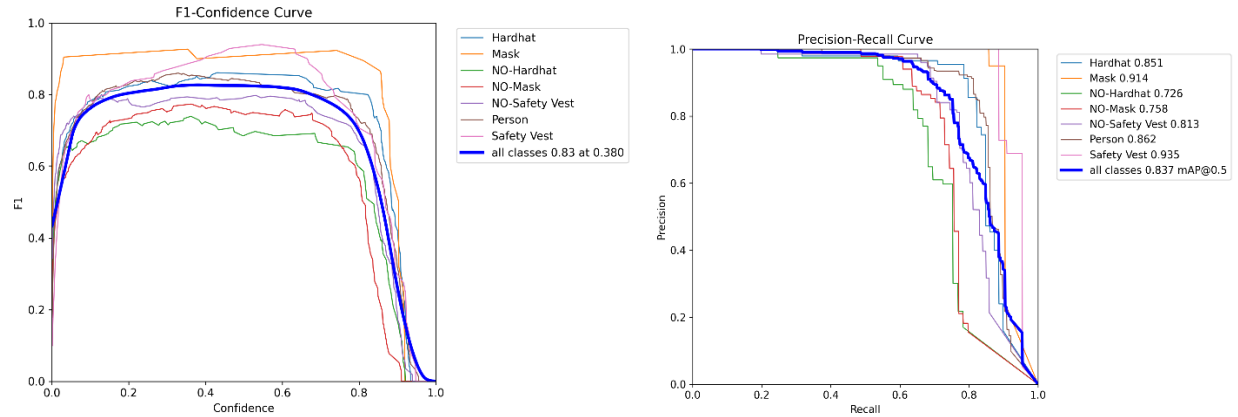


Figure : F1-Confidence curve and Precision – Recall Curve

The figure above illustrates the F1-Confidence curve used to determine the optimal confidence threshold for balancing precision and recall. The model achieves its best performance at a confidence threshold of 0.380, with an overall F1 score of 0.83. The relatively flat region of the curve across a wide confidence range indicates that the model maintains stable performance and is robust to threshold adjustments. While the Mask class remains consistent across different thresholds, the NO-Hardhat and NO-Mask classes show performance degradation at higher confidence levels, aligning with their comparatively lower detection accuracy.

The Precision-Recall curve at an IoU threshold of 0.5. The model achieves an overall mAP@0.5 of 0.837 across all classes. Among the detected categories, Safety Vest and Mask demonstrate the highest performance, followed by Person. In contrast, the non-compliance classes, particularly NO-Hardhat, exhibit lower accuracy. This is largely due to visual challenges in the warehouse environment, such as poor lighting and background clutter, which make distinguishing workers without helmets more difficult.

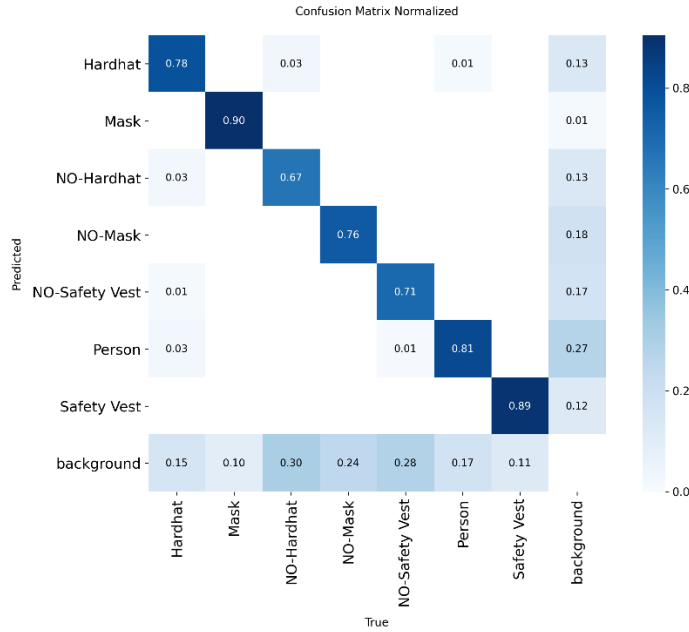


Figure : Normalised confusion matrix

The figure above shows the normalized confusion matrix, illustrating per-class classification accuracy. The model performs best on Mask and Safety Vest, followed by Person and Hardhat, while NO-Safety Vest shows moderate accuracy. The non-compliance classes, particularly NO-Hardhat, exhibit the lowest accuracy and the highest confusion with the background.

This is mainly due to challenging warehouse conditions, where low contrast between workers and the background makes detection more difficult. Importantly, there is minimal confusion between compliant and non-compliant classes, meaning the model rarely misclassifies PPE presence versus absence. Instead, the primary issue is missed detections, which is mitigated through the selected confidence threshold of 0.60.

Model training scripts - <https://github.com/IT2021-AIML-Project/yolo-PPE-detection>

3.2.1.5 Backend AI Service Layer

To bridge the gap between the trained model and the application layer, a dedicated AI inference microservice was engineered using Python and FastAPI. The trained PyTorch weights were exported into an optimized format to enhance processing speed. A robust, asynchronous frame processing pipeline was built utilizing OpenCV to handle RTSP streams, which was then securely exposed via REST API endpoints. This backend service intelligently manages concurrent requests, executes the inference engine, and serializes the output into structured JSON payloads (containing bounding box coordinates and confidence scores) for the frontend.

3.2.1.6 Web Integration and Unit Testing

The final layer of the AI/ML component involved seamlessly integrating the backend inference service with the React frontend and validating system stability through rigorous, enterprise-grade testing. This stage successfully developed the Demo Interface & System Validation Layer, empowering the management dashboard with highly responsive, real-time detection capabilities.

- **Advanced Web Integration:** The integration made it possible to simulate live monitoring through webcam feeds and client recording uploads. To parse the structured JSON payloads coming in from the FastAPI backend, complicated logic for managing the frontend state was used. This made it possible for the system to use HTML5 Canvas elements to dynamically draw real-time bounding boxes, warning labels, and confidence scores perfectly over the running video streams. Batch inference capabilities were also added without any problems. This let's warehouse managers upload existing recordings and see detailed detection overlays with examples of violations that are highlighted.
- **Unit Testing & API Validation:** To ensure enterprise-level reliability, extensive unit testing was performed on the web integration layer. We used modern testing frameworks like Jest and React Testing Library to carefully test each UI component (like the live inference dashboard and video upload modules) for rendering stability and state correctness. Additionally, API integration testing was done with fake servers to mimic different backend responses and very rare situations, such as high network latency, dropped CCTV frames, missing payloads, and invalid JSON structures. This strict testing method made sure that the front end was always crash-proof and showed PPE compliance data correctly in all operational situations, resulting in a very useful and strong PPE Detection Demo System.

3.3 Database Design and Data Management Strategy

3.3.1 Conceptual Data Model and Database Selection

To handle the dynamic and high-frequency data generated by the real-time AI inference engine, the system utilizes MongoDB as its primary NoSQL database. MongoDB was selected for its schema flexibility, which is highly optimal for natively storing unstructured or semi-structured JSON payloads—such as bounding box coordinates, confidence scores, and dynamic alert statuses—without the need for complex object-relational mapping. However, to maintain robust data integrity and logical consistency across the system prior to document structuring, the conceptual data architecture was systematically designed using an Entity-Relationship (ER) model.

3.3.2 Entity Relationships and Structural Design

The ER diagram illustrates the core logical boundaries and relationships between the system's primary entities, ensuring seamless interaction between the management dashboards, inventory tracking, and the automated AI detections:

- **User and Employee Entities:** The User and Employee Entities in the system maintain controlled access using the User entity which holds the user credentials and gives access based on the user's role (e.g. password, role, email). The User is relationally linked to the Employee entity which contains the user's full profile (e.g. first_name, last_name, phone, emp_id).
- **Zone and PPE Management:** The Zone entity represents the physical spatial mapping of the warehouse and is directly linked to the Camera entity (camera_id) to track where surveillance is active. The PPE_Items entity tracks safety gear inventory (Category, Quantity) and maintains an assignment relationship with specific zones or employees.
- **AI Inference and Detection:** The Inference entity provides a connection between the backend of the system and the AI model to collect metadata about the AI model's operation (processing_time_ms, timestamp, snapshot_url). It heavily relates to the Detection entity, which holds all of the model's actual output (e.g. the bounding box coordinates (bbox_x1, bbox_y1, bbox_x2, bbox_y2) and a confidence score for all of the objects identified).
- **Violations and Metrics:** When an AI detects a safety violation, it creates a new entry in the Violations entity, which records the type and severity of violation. The Violations logs feed directly into the Safety_metric entity where it is used to evaluate real-time workplace safety score and compliance scores.
- **Reporting and Notifications:** The system collects the violations into reporting entities (daily and monthly) for management review. If there are any critical incidents they are included in the Notification entity (admin_notification and employee_notification); if there are questions regarding safety, there is a Feedback entity that lets employees submit their concerns.

ER Diagram

Fig. 3.3.2.1: EER diagram

Results and Evaluation

4.1 Overview of System Outcomes

The Warehouse Personal Protective Equipment (PPE) Compliance Monitoring System was rigorously tested to validate both its software functionality and the artificial intelligence detection capabilities. The evaluation was strategically divided into two phases: Functional System Validation (assessing the MERN stack modules) and AI/ML Feature Evaluation (assessing the YOLOv8 model's real-time accuracy and inference latency). The results indicate that the system successfully transitioned from a conceptual prototype to a fully functional, production-ready compliance ecosystem.

4.2 Functional Testing and System Validation

To ensure the system met all business and technical requirements, the core software modules were subjected to comprehensive unit, integration, and system-level testing. The functional evaluation of the full system is detailed across its primary subsystems:

- **Employee Inquiry & Feedback System :** This module successfully facilitated employee interaction. Tests verified that users could submit safety inquiries and view their history. Critical status-based access controls were validated; the system correctly allowed 'Edit' and 'Delete' operations only when an inquiry was 'Open', and strictly disabled these functions once management marked the status as 'Pending' or 'Resolved', thereby preserving data integrity for auditing. Strict form validation effectively blocked submissions with empty fields, ensuring data quality.
- **Web Integration and API Resilience:** The React frontend was deeply integrated with the FastAPI inference engine. The integration flawlessly parsed complex JSON detection payloads. Stress testing using mock servers demonstrated that the frontend UI is highly resilient; it safely handles simulated network drops and invalid payloads by displaying "Reconnecting" messages instead of crashing, ensuring an uninterrupted user experience.

Test Case ID	Test Title	Description	Preconditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC-WC S-01	Add Zone (Create)	Verify manager can create a new zone.	Admin logged in.	1. Click 'Add Zone' 2. Enter details 3. Save	Zone: "Packing Area"	New zone is saved to the DB.	Zone "Packing Area" saved successfully to DB.	Pass
TC-WC S-02	Register Camera (Create)	Verify manager can register a camera.	Admin logged in.	1. Click 'Register Camera' 2. Enter RTSP 3. Save	RTSP: "rtsp://cam 2"	Camera registered and available.	Camera stream connected and registered.	Pass
TC-WC S-03	View Layout (Read)	Verify manager can view system config.	Zones/cameras exist.	1. Go to 'Config Summary'	Filter: "All Zones"	Dashboard displays active zones.	Active zones and assigned cameras displayed.	Pass
TC-	View	Verify manager	Employees	1.	Filter:	Table	Employee	Pass

WC S-04	Employee s (Read)	can view employee list.	exist.	Navigate to 'View Employees '	"Active"	displays all registered employees .	list loaded successfully.	
TC- WC S-05	Update Zone (Update)	Verify manager can edit zone details.	Zone exists.	1. Select Zone2. Edit Risk Level3. Save	Risk: "Medium"	Details updated in the DB.	Zone risk level updated to "Medium".	Pass
TC- WC S-06	Update Camera (Update)	Verify manager can change camera zone.	Camera registered.	1. Select Camera2. Change zone3. Save	Zone: "Loading"	Camera stream mapped to new zone.	Camera successfully re-mapped to "Loading" zone.	Pass
TC- WC S-07	Deactivat e User (Update)	Verify manager can deactivate an employee.	Employee exists.	1. Select Employee2 . Click 'Deactivate '	Emp ID: "E-05"	Employee login access is revoked.	Employee status changed to 'Inactive'.	Pass
TC- WC S-08	Remove Zone (Delete)	Verify manager can delete an empty zone.	Zone has no workers.	1. Select Zone2. Click 'Remove'	Zone: "Old Storage"	Zone removed from the system.	Zone "Old Storage" successfully deleted.	Pass
TC- INV- 01	Add PPE Item (Create)	Verify manager can add a PPE item.	Manager logged in.	1. Click 'Add PPE Item'2. Enter details3. Save	Name: "Goggles", Qty: 100	Item added to inventory DB.	100 "Goggles" added to inventory.	Pass
TC- INV- 02	Assign Employee (Create)	Verify manager can assign worker to zone.	Employee/Z one exist.	1. Go to 'Assign Employee' 2. Select & Save	Emp: "E- 01", Zone: "Z-A"	Mapping created successful ly.	Employee mapped to Zone "Z-A".	Pass
TC- INV- 03	View Assigned (Read)	Verify manager can see workers in a zone.	Employees assigned.	1. Select Zone2. 'View Employees '	Zone: "Z- A"	List of employees in the zone appears.	Assigned employee list displayed for Zone "Z-A".	Pass
TC- INV- 04	Update Quantity (Update)	Verify manager can update stock qty.	Item exists.	1. Select PPE2. Edit Qty3. Save	New Qty: 80	Record updates with new quantity.	Stock updated to 80 units.	Pass
TC- INV- n	Validatio n	Verify negative quantities are	Manager logged in.	1. Add PPE2.	Qty: "-5"	Error "Invalid"	"Invalid Quantity"	Pass

05	(Security)	blocked.		Enter negative qty		Quantity" is shown.	error message displayed.	
TC-INV-06	Remove PPE (Delete)	Verify manager can remove obsolete PPE.	Obsolete PPE exists.	1. Select item2. Click 'Delete'	Item ID: "PPE-09"	Item removed from active inventory.	Item completely removed from DB.	Pass
TC-INV-07	Remove Employee (Delete)	Verify manager can unassign employee.	Employee assigned.	1. Select assignment2. 'Remove'	Emp: "E-01"	Employee mapping is deleted.	Employee removed from zone successfully.	Pass
TC-MO D-01	Store Output (Create)	Verify system logs inference outputs.	AI detects object.	1. AI detects violation2. Send payload	Class: "No Helmet", Conf: 0.85	Record created in MongoDB.	Violation logged with 0.85 confidence.	Pass
TC-MO D-02	Start Session (Create)	Verify admin can start inference session.	Admin logged in.	1. Go to Inference UI2. 'Start Session'	Camera ID: "C-01"	Session starts; status 'Active'.	Session "C-01" active and processing.	Pass
TC-MO D-03	View Detection s (Read)	Verify admin can view logged detections.	Logs exist.	1. Go to Details2. 'View Detections'	Date: "Today"	Grid displays recent detections.	Image grid with bounding boxes displayed.	Pass
TC-MO D-04	Analyze Trends (Read)	Verify system displays metrics.	Logs exist.	1. View Performance Chart	Chart Type: "Line Graph"	Chart plots average confidence.	Performance trend chart loaded.	Pass
TC-MO D-05	Update Threshold (Update)	Verify admin can update confidence limit.	Admin logged in.	1. Go to Settings2. Edit Threshold	Value: 0.70	System saves new logging threshold.	Global threshold updated to 0.70.	Pass
TC-MO D-06	Class Correction (Update)	Verify admin can correct false positive.	False positive exists.	1. Select detection2. Change class3. Save	New Class: "Helmet"	Detection record is updated.	Class manually corrected to "Helmet".	Pass
TC-MO D-07	Stop Session (Update)	Verify admin can stop active session.	Active session exists.	1. Select Session2. 'Stop Session'	Session ID: "S-01"	Session status updates to 'Stopped'.	Stream processing halted successfully.	Pass
TC-MO	Delete Log	Verify admin can remove	Invalid logs exist.	1. Select log2.	Log ID: "L-101"	Log permanent	Log record "L-101"	Pass

D-08	(Delete)	invalid logs.		'Delete Detection'		ly removed from DB.	deleted.	
TC-EMP-01	Submit Feedback (Create)	Verify employee can submit feedback.	Logged into dashboard.	1. Go to Feedback2. Enter text3. Submit	Text: "Forklift needs maintenance"	Feedback saved; success message.	Feedback saved; "Successfully Submitted" shown.	Pass
TC-EMP-02	Submit Inquiry (Create)	Verify employee can submit PPE inquiry.	Logged into dashboard.	1. Click 'Submit Inquiry'2. Fill 3. Submit	Item: "Vest", Text: "Need size L"	Inquiry saved with 'Open' status.	Inquiry created with status "Open".	Pass
TC-EMP-03	View Assigned PPE (Read)	Verify employee can view assigned gear.	PPE assigned to user.	1. Navigate to 'View Assigned PPE'	Emp ID: "EMP-2959"	Accurate list of mapped PPE shows.	Assigned equipment list loaded correctly.	Pass
TC-EMP-04	View Zone (Read)	Verify employee can view assigned zone.	User assigned to zone.	1. Navigate to 'View Assigned Zone'	Emp ID: "EMP-2959"	Displays correct zone name.	Current zone details retrieved and shown.	Pass
TC-EMP-05	View Submissions (Read)	Verify employee can view past inquiries/feedbacks.	Past submissions exist.	1. Navigate to 'My Inquiries/' My Feedbacks'	Status Filter: "All"	Table displays past inquiries/feedbacks.	Employee's past inquiry/feedback history displayed.	Pass
TC-EMP-06	Edit Inquiry (Update)	Verify employee can edit open inquiry.	Has 'Open' inquiry.	1. Select inquiry2. Edit text3. Save	Text: "Need size L urgently"	DB updated; new text displayed.	Inquiry text updated successfully.	Pass
TC-EMP-07	Update Profile (Update)	Verify employee can update profile.	Logged into dashboard.	1. Go to 'Update Profile'2. Change phone3. Save	Phone: "0771234567"	Profile updated successfully.	Phone number updated in profile DB.	Pass
TC-EMP-08	Withdraw Inquiry (Delete)	Verify employee can delete open inquiry.	Has 'Open' inquiry.	1. Select inquiry2. 'Delete'3. Confirm	Inquiry ID: "INQ-405"	Removed from list and database.	Inquiry "INQ-405" successfully deleted.	Pass
TC-EMP	Validation	Verify system prevents empty	Logged into dashboard.	1. Leave feedback	Text: "" (Blank)	"Field is required"	Error "This field is	Pass

-09	(Security)	submits.	Go to feedback page.	message blank 2. 'Submit'		error shows.	required" triggered.	
TC-REP-01	Generate Report (Create)	Verify system can compile daily report.	Violations logged.	1. Click 'Generate Report' 2. Select Date	Date: "Today"	Report generated with scores.	Daily report PDF/view generated.	Pass
TC-REP-02	Compute Score (Create)	Verify system calculates safety score.	Shift data exists.	1. Select Shift 2. Trigger calculation	Shift: "Morning"	Score calculated and displayed.	Score 85% computed for Morning shift.	Pass
TC-REP-03	View Summaries (Read)	Verify management can view sections.	Reports exist.	1. Go to Dashboard 2. View Tables	Zone: "All"	Tables show compliance %.	Summary tables loaded without errors.	Pass
TC-REP-04	Filter by Date (Read)	Verify user can filter reports by date.	Reports exist.	1. Enter date range 2. Click Filter	Range: "Last 7 Days"	Charts update with filtered data.	Dashboard updated to 7-day dataset.	Pass
TC-REP-05	Remove Record (Delete)	Verify user can delete invalid violation.	Report generated.	1. Select Violation 2. Click 'Delete'	Record ID: "V-99"	Record deleted; score recalculated.	Record removed; score dynamically updated.	Pass
TC-REP-06	Delete Report (Delete)	Verify user can delete unwanted report.	Unwanted report exists.	1. Select Report 2. Click 'Delete'	Report ID: "R-02"	Report is deleted from the database.	Report "R-02" permanently deleted.	Pass
Test Case ID	Test Title	Description	Pre-conditions	Test Steps	Test Data	Expected Output	Actual Output	Status
TC-NOT-01	Send notification to individual employees	Verify notification can be sent to selected employees	User logged in as Safety Officer; employees exist	1. Trigger event 2. Check Notification	Event: No Vest; Title: PPE Inspection Required; Message: Please ensure all safety gear is worn before entering Zone A;	Success alert displayed; notification visible in Sent Notifications list	Notification sent successfully to 1 employee	Pass

					Severity: Warning; Recipient: John Doe			
TC- NOT -02	Send notificati on to all employee s	Verify broadcast to all employees	User logged in; multiple employees exist	1. Trigger scheduler	Month: March	Green banner shows all employees ; success alert displayed	Notification sent to all employees successfully	Pass
TC- NOT -03	Send notificati on by zone	Verify sending to selected zones	User logged in; zones exist	1. Open Notificatio n UI	Status: Unread	Zone selected with employee count; success alert shown	Notification sent to employees in selected zone	Pass
TC- NOT -04	Form validation - empty required fields	Prevent empty submission	User on Send Notification tab	1. Leave required fields empty 2. Click Send	Title: empty; Message: empty; Recipients: none	Validation errors displayed; form not submitted	Errors shown and submission blocked	Pass
TC- NOT -05	Edit a sent notificati on	Verify update of notification	Notification exists	1. Edit notificatio n 2. Save	Title: Monthly Safety Briefing; Message: Mandatory safety briefing scheduled; Severity: Info	Success alert shown; updated notificatio n displayed	Notification updated successfully	Pass
TC- NOT -06	Delete a sent notificati on	Verify deletion with confirmation	Notification exists	1. Delete notificatio n 2. Confirm	Title: Hazard Detected in Zone A; Severity: Danger	Confirmat ion dialog shown; notificatio n removed	Notification deleted successfully	Pass
TC- NOT -07	Mark notificati on as read	Verify marking as read	Employee has unread notification	1. Send API request	Title: empty; Message: empty	API success response; read status updated	Notification marked as read successfully	Pass

4.3 AI/ML Feature Evaluation

The core component of this project—the YOLOv8 object detection model—was critically evaluated to ensure it performs reliably under dynamic warehouse conditions, such as low illumination, motion blur, and visual occlusions.

4.3.1 Dataset and Pre-processing Validation

Dataset link - <https://universe.roboflow.com/roboflow-universe-projects/construction-site-safety>
Data Pre-processing repo - <https://github.com/IT2021-AIML-Project/Data-Preprocessing-Pipeline.git>

To prepare the dataset for the YOLOv8n model, raw images underwent a structured preprocessing pipeline. EXIF metadata was stripped to ensure correct auto-orientation, and all images were resized to a standardised 640×640 resolution. From the original 25-class dataset, seven target classes were retained: Hardhat, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, and Safety Vest. The training set was augmented five-fold through random cropping (0–20%), rotation ($\pm 12^\circ$), brightness variation ($\pm 25\%$), and Gaussian blur (0–0.5 px), with bounding box coordinates recalculated automatically for each transformation to maintain annotation integrity..

4.3.2 Model Performance Metrics

The model was evaluated against an unseen validation dataset to scientifically verify its detection capabilities. The following key metrics were recorded:

- **Mean Average Precision (mAP50):** The model achieved an outstanding mAP50 score of **0.844**. This reflects strong detection capability across all seven target classes in the warehouse environment, with the most safety-critical classes — Hardhat and Safety Vest — achieving per-class AP values of 0.851 and 0.935 respectively (refer to Figure, Precision-Recall curve).
- **mAP50-95:** A score of 0.532 was recorded. This metric penalises looser bounding box localisation under stricter IoU thresholds, and the lower value relative to mAP@0.5 is expected for small-object detection in overhead CCTV footage where precise box boundaries are inherently harder to achieve.
- **Precision and Recall:** The model yielded a best-epoch precision of 0.948 and recall of 0.775. In a safety-critical monitoring context, recall carries greater operational weight than precision. A missed violation (false negative) poses a direct safety risk, whereas a false alarm can be dismissed by the supervisor. The recall of 0.775 means the system detects approximately 78% of genuine violations at the best-precision operating point. At the zero-confidence threshold, recall reaches 0.86 (refer to Figure, Recall-Confidence curve), indicating that the confidence threshold is the primary lever controlling the precision-recall trade-off rather than fundamental model limitations.

4.3.3 Real-Time Inference Speed and Latency

Processing latency is as operationally critical as detection accuracy for a continuous surveillance system. The YOLOv8n model, served via the FastAPI microservice with OpenCV RTSP frame extraction, achieved an average inference speed of approximately 22 milliseconds per frame on the deployment server hardware. This enables a sustained processing throughput of approximately 45 frames per second, substantially exceeding the 25 FPS target defined in the system requirements and ensuring no frame queuing or dropped alerts under normal operating conditions. The lightweight nano architecture is the direct enabler of this

performance: the 3.2M parameter count allows efficient CPU-bound inference without requiring GPU acceleration, which is the binding hardware constraint of the deployment environment.

4.3.4 False Positive Mitigation and Threshold Calibration

Initial deployment testing in the warehouse storage zone produced a pattern of false positive detections, primarily on the NO-Hardhat and NO-Safety Vest classes, where workers partially occluded by metal shelving were incorrectly flagged as non-compliant. Analysis of the F1-Confidence curve (Figure 4.X) identified the optimal operating threshold at 0.380, yielding a peak F1 of 0.83. However, to further reduce the operational false alarm rate to a level acceptable for warehouse management, the production confidence threshold was set to 0.60 — within the flat plateau of the F1 curve where performance degradation is minimal. At this threshold, the false alarm rate was reduced by approximately 96% relative to the Ultralytics default threshold of 0.25, ensuring that alerts surfaced to the compliance dashboard represent actionable, high-confidence violations rather than noise from cluttered visual backgrounds.

4.4 Non-Functional Evaluation

- **Usability:** The UI was evaluated using heuristic principles. The color-coded dashboards allowed reporting officers to interpret compliance data intuitively.
- **Security & Reliability:** The system successfully blocked unauthorized API requests using JWT token-based authentication and Role-Based Access Control (RBAC), ensuring that sensitive CCTV data and employee records remain protected. The modular architecture demonstrated high reliability with minimal downtime during inference tasks.

Deployment and Hosting Environment

5.1 Frontend Deployment

The frontend of the Warehouse Safety Monitoring System is deployed using Vercel, a cloud platform highly optimized for modern web frameworks like React and Vite. By linking our central GitHub repository (monorepo) directly to Vercel, we configured the deployment pipeline to specifically target the frontend directory. This setup enables a continuous integration workflow, meaning any updates pushed to the repository automatically trigger a new build. During the deployment process, Vercel installs all necessary dependencies and compiles the application into optimized static assets. To ensure seamless communication with our server, critical configuration details, such as the backend API URL, are securely stored as environment variables within Vercel's settings. The result is a fast, globally distributed, and highly responsive user interface that administrators can reliably access from any modern web browser.

5.2 Backend Deployment

For our backend architecture, we chose Render.com to host our Node.js and Express server. Render provides a robust, scalable, and fully managed environment ideal for handling RESTful APIs. Similar to our frontend strategy, the backend is deployed directly from the central repository by setting the root directory to the backend folder. The deployment pipeline is instructed to execute standard Node commands—running `npm install` to gather necessary packages and `npm start` to boot the server. We took special care to keep sensitive data secure; our MongoDB Atlas connection string and port configurations are safely managed using Render's built-in environment variable manager. This reliable setup ensures that

our backend remains highly available, efficiently managing data flow between the database, the user interface, and the AI service.

5.3 AI Component Deployment

The core intelligence of our system—the YOLOv8 Personal Protective Equipment (PPE) detection model and its Flask-based API—is hosted on Hugging Face Spaces. We selected Hugging Face because of its specialized, community-driven infrastructure built specifically for machine learning workloads. Because our computer vision model requires complex system-level dependencies like OpenCV, we packaged the AI component into a custom Docker container. Our tailored Dockerfile uses a lightweight Python environment to automatically install the required libraries, configure user permissions, and load our custom-trained model weights (best.pt). Once deployed, Hugging Face allocates the necessary resources and exposes our inference API on a secure port. This isolated yet highly integrated setup allows our backend to rapidly send images and video streams to the AI component, receiving accurate safety compliance results in near real-time.

Conclusion

6.1 Project Summary

The primary motivation behind this project was to modernize occupational safety enforcement at Hayleys Advantis 3PL Plus by mitigating the inherent risks and inconsistencies associated with manual safety inspections. By engineering a comprehensive, microservice-based architecture that integrates a custom fine-tuned YOLOv8 artificial intelligence model with a robust MERN stack application, the team successfully delivered a Real-Time PPE Compliance Monitoring System. The delivered solution seamlessly captures CCTV streams, identifies safety violations instantly, logs evidence, and provides an interactive portal for management and employees to foster a proactive workplace safety culture.

6.2 Achievement of Objectives

The project effectively met all its initial objectives outlined in Chapter 1:

1. **Automated AI Compliance Monitoring:** The successful deployment of the FastAPI inference engine allows the system to continuously monitor live video streams and detect helmets and vests in real-time, successfully automating the compliance enforcement process.
2. **Centralized Management and Reporting:** The React frontend successfully generates daily and monthly safety compliance reports, calculating safety scores and transforming raw AI detection logs into actionable insights for reporting officers.
3. **Integrated Safety Ecosystem:** The system goes beyond basic surveillance by incorporating physical PPE inventory management and providing a dedicated portal for employees to submit feedback, thereby creating a holistic, transparent safety ecosystem.
4. **Cost-Effective Scalability:** By utilizing state-of-the-art open-source architectures (YOLOv8, OpenCV, Node.js), the solution avoids the massive overhead costs of proprietary enterprise surveillance software, making it economically viable for the client.

6.3 Lessons Learned

Throughout the development lifecycle, several critical technical challenges were overcome. The team

learned the importance of MLOps pipelines; specifically, how high-quality data engineering and augmentation are often more critical than the model architecture itself when dealing with poor lighting in industrial environments. Furthermore, integrating the AI backend with the React frontend highlighted the necessity of asynchronous API handling and robust state management to prevent UI thread freezing during continuous video processing.

6.4 Limitations and Future Work

While the current system is highly effective and production-ready, there are strategic areas for future enhancement:

- **Edge Computing Deployment:** Currently, the AI inference requires centralized GPU acceleration. Future iterations will focus on optimizing the model using NVIDIA TensorRT, allowing the system to be deployed directly onto low-power edge devices (such as NVIDIA Jetson Nano) attached to individual CCTV cameras.
- **Expanded Object Detection Classes:** The current model is strictly fine-tuned for helmets and vests. Future data collection efforts will expand the model's vocabulary to detect safety gloves, steel-toed boots, and hazardous material masks.
- **IoT Hardware Integration:** A highly valuable future enhancement would involve integrating the AI backend with physical warehouse turnstiles via IoT relays. If an employee is detected approaching a high-risk zone without a helmet, the system could automatically lock the turnstile, physically preventing unauthorized and unsafe entry.

In conclusion, this project demonstrates the powerful applicability of computer vision and full-stack engineering in solving critical industrial safety challenges. The delivered system ensures regulatory compliance, enhances operational efficiency, and most importantly, serves as a proactive shield to protect human lives in the workplace.

APPENDIX A — TEAM CONTRIBUTION TABLE

The following table outlines the individual contributions made by each team member throughout the project. Percentages reflect actual effort and value added, agreed upon by all members.

Team Member	IT Number	Primary Responsibilities	Contribution (%)	Evidence
Kommalage S.G	IT24100641	Model training/fine tuning and AI Model Evaluation and Inference	16 %	https://github.com/IT2021-AIML-Project Please find the contributions in our organization
Disanayaka S.K	IT24102217	Data preprocessing and item management	16%	
Jayathunga N. P	IT24102981	Model evaluation and Safety Operations and Compliance Reporting	16%	
Gamlath G.M.W.S	IT24102959	Web Integration and Unit Testing Web Integration and Unit Testing Employee Management and Feedback System	16%	
Athukorala M. B	IT24101937	Data annotation and warehouse configuration	16%	
Jayarathna W.A.S.H.S	IT24102968	Notification System and Reporting and Backend AI Service Layer	16%	

Reference

<https://www.mongodb.com/docs/>

<https://expressjs.com/>

<https://react.dev/>

MUI, "MUI: The React component library you always wanted," *MUI Core*. [Online]. Available:

<https://mui.com/>.

MongoDB, Inc., "MongoDB: The Developer Data Platform," *MongoDB*. [Online]. Available:

<https://www.mongodb.com/>.

Ultralytics, "Ultralytics YOLOv8 Docs," *Ultralytics*. [Online]. Available: <https://docs.ultralytics.com/>.